Studio Nx

AI 구현 설명 49 — P1-13 프로젝트 API

2026-10-08

AI 구현 설명 49 — P1-13 프로젝트 API (만들기 · 목록 · 상세 · 수정 · 삭제 · 권한)

항목	내용
작성	Claude (AI) · 2026-10-08
작업	P1-13 프로젝트 CRUD · 권한 검사 — PRD 01 §5 · §7 · FR-122~126 · FR-130
브랜치	`feature/phase1/p1-13-project-api`
바꾼 파일	백엔드 `schemas/project.py` · `services/project_service.py` · `api/v1/projects.py` / 시험 `test_project_schemas.py`(새) · `test_project_service.py`(새) · `test_project_routes.py`
진행 방식	**함수 하나씩** 구현 → 사용자 검토 → 다음
함수별 설명	`docs/ai-notes/backend(fastAPI_Python)/P1-13 - 프로젝트 API/56 ~ 62` (md · PDF), 합본 P1-13_project_api.pdf
실제 사이트 시험	50_p1-13_site_test.md — 실제 서버에 5개 API, 26가지 모두 통과
이전 문서	48_p1-12_site_test.md

1. 한 일

myProject.html(P1-14)이 쓸 **프로젝트 API 5개**가 실제로 동작한다.

요청	하는 일	성공	권한
`GET /api/v1/projects`	내 프로젝트 목록 — 검색 · 정렬(updated · title · created) · 페이지	200	로그인
`POST /api/v1/projects`	만들기 — 프로젝트 + OWNER 멤버 + MASTER 트랙을 한 트랜잭션으로	201	로그인
`GET /api/v1/projects/{id}`	상세	200	멤버
`PATCH /api/v1/projects/{id}`	제목 · 설명 수정 (보낸 칸만)	200	OWNER · EDITOR
`DELETE /api/v1/projects/{id}`	소프트 삭제 (`deleted_at`)	204	OWNER

2. 사용자 결정 (2026-10-08)

#	정한 것	결정	템플릿
1	남의 프로젝트에 접근	**A — 404 "Project not found."** (없는 번호 · 삭제된 것과 같은 응답). 멤버인데 역할이 모자라면 403	404 권장

3. 항목 (백엔드 7)

#	파일	함수	하는 일	설명
56	`schemas/project.py`	`ProjectCreateRequest` · `ProjectUpdateRequest`	제목 · 설명 공백 제거 후 검사, 빈 설명 → None, 박자 아래 1·2·4·8·16·32, 수정에서 제목 null → 422	56
57	`services/project_service.py`	`get_membership()` · `require_role()`	프로젝트 + 내 멤버 행 (없으면 404) / 역할 검사 (403)	57
58	〃	`to_item()` · `list_projects()`	화면 모양으로 바꾸기 (my_role · member_count · opened) / 목록 — 쿼리 4번, 검색어 escape	58
59	〃	`create_project()`	세 행을 한 트랜잭션, 실패하면 rollback	59
60	〃	`update_project()`	`exclude_unset` 으로 보낸 칸만, `updated_at` 은 DB	60
61	〃	`delete_project()`	OWNER 만, `deleted_at` = 지금(UTC)	61
62	`api/v1/projects.py`	라우터 5개	서비스 연결 · 201 · 204	62

4. 템플릿에서 달라진 점

항목	템플릿	구현	이유
제목 공백	`min_length=1` + 서비스에서 `strip()`	스키마에서 공백 제거 후 검사	공백 한 칸짜리 제목 막기 · 문의(50)와 같음 (56)
제목 null → 422	서비스에서	스키마에서	다른 검사 오류와 같은 모양 (56)
빈 설명	—	None 으로	`""` 와 NULL 이 섞이지 않게 (56)
`to_item`	프로젝트마다 계산	값을 넘기면 그대로, 안 넘기면 DB 에 물어봄	목록은 모아서 계산 (N+1 방지), 상세 · 수정은 그대로 (58)
`opened`	P1-16 전엔 False 로 둬도 됨	`project_sessions` 를 실제로 봄	표가 이미 있음 — Phase 2 에서 기록이 생기면 바로 맞게 나옴 (58)

5. 시험

방법	대상	결과
pytest (실제 MySQL · 실제 앱)	56 ~ 62	`test_project_schemas.py` 33 · `test_project_service.py` 42 · `test_project_routes.py` 13 → **전체 492개 통과**, `ruff` 통과
실제 서버 + 브라우저 (50)	5개 API · 두 회원(주인 · 남/VIEWER)	**26가지 모두 통과**, DB 확인 (MASTER 트랙 · OWNER/VIEWER 행 · `deleted_at`)

시험에서 알게 된 것: 응답의 `bpm` 은 숫자가 아니라 **글자** `"128.50"` (pydantic 의 Decimal 규칙). P1-14 화면에서 숫자가 필요하면 `Number(item.bpm)`.

6. 다음 작업 · 남은 것

작업	내용
P1-14 내 프로젝트 화면 JS	`myproject.js` — `loadProjects` · `renderProjects` · `renderPager` · 검색 · 새 프로젝트 · 수정 · 삭제 (`confirmDialog`) · `initLogoutButton` import · `requireLogin` (함수 하나씩)
P1-16 대시보드 API	`opened` · 최근 작업 시간 (`project_sessions`)
Phase 6	멤버 초대 — 지금은 OWNER 한 행만 생김, 역할 검사는 이미 준비됨